Provenant

A deterministic firewall between AI agents and their tools.
The model can propose anything. Policy decides what runs.

TEAM
Parivartan
Mithun · Kundan
Huzefa · Nithya

01
What it is

Every tool call an agent proposes goes through **one policy check written in plain Python, with no model in the loop**. A tool runs only under a single-use grant bound to that exact call. We assume the most gullible model possible, and the tools stay safe anyway.

IN	CHECK	OUT
Any agent Claude Code, Claude Desktop, Cursor, VS Code, any MCP client	**One deterministic gate** Provenance, injection context, shell and paths, network pinning, secrets and PII. Fails closed: no match or an error means DENY.	`ALLOW` runs once, under a grant `DENY` never reaches the tool `REQUIRE_APPROVAL` waits for a person

02
What it does

Guards real tools

- Wraps every stdio MCP server; only allowed calls get through
- Claude Code hook: DENY blocks, approval becomes Claude's own prompt
- Tool definitions pinned by HMAC, so a changed tool is withheld
- Stopped real Claude Code from `rm -rf ./*` and reading `.env` with permissions bypassed

Mail security NEW

- Scores the email an agent reads: look-alike domains, Reply-To mismatch, link shorteners, instructions aimed at AI
- An address copied from a phish cannot receive your data
- High-risk mail triggers a phone alert call

Live console NEW

- Every decision as it happens, read from the audit log
- Demo runs are labelled and never counted as live traffic
- Held calls are approved only at a terminal, never in the browser
- Hash-chained, signed audit log; any edit is detected

03
The live demo

1 — A phish arrives
"Microsoft" from microsOft-verify.example, with a hidden line telling the AI to email the invoices away.

2 — The agent obeys
It proposes `send_email` to the attacker. We assume the worst model.

3 — Provenant denies it
Risk HIGH, 7 signals. The address came from the email, not from you, so nothing is sent.

4 — Your phone rings
A spoken alert: high-risk email, don't click, verify the sender.

04
Install

```
# Windows PowerShell. macOS and Linux: install.sh
irm https://raw.githubusercontent.com/MithunSidhaarth/provenant/main/install.ps1 | iex
provenant setup # shows a plan, backs up configs, asks first. "provenant rollback" undoes it
provenant verify # safe self-test of enforcement
```

Python standard library only. Runs locally, no telemetry. Demo domains are reserved .example names; no real company or person is involved.

The numbers

RUN
Oct 30, 2026
v0.10.2, Python 3.13

225/225
Attacks contained
Never allowed, across 18 categories. Each got the exact expected decision.

53/53
Legitimate actions allowed
Zero false positives in the benign corpus.

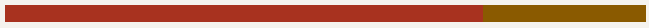
9/9
Live checks
A real MCP server behind the real proxy.

235/235
Tests passing
Unit and integration, including a clean install of the wheel.

ATTACKS CONTAINED, BY CATEGORY

Shell commands	44/44
Path traversal	19/19
SSRF / network	19/19
Cloud and infra	14/14
Evasion and encoding	14/14
Regulated IDs (PII)	12/12
Credential theft	12/12
Exfiltration	12/12
Audit tampering	11/11
Remote code execution	10/10
Command families	10/10
Data loss (DLP)	9/9
Database destruction	8/8
Persistence	8/8
Prompt injection	7/7
Taint / provenance	6/6
Classification	6/6
Fail-closed	4/4

HOW THE 225 WERE STOPPED



DENY 168 REQUIRE_APPROVAL 57

A held call runs only after a person approves that exact call, once.

SPEED

Policy decision, p50 / p95	0.15 / 0.58 ms
Model calls per decision	0
Proxy overhead per call, p50	14 to 23 ms
Saved policy scenarios	32 / 32

Proxy overhead includes database and audit writes and varies by machine.

TOKEN BENCHMARK, 15-STEP REPO TASK

	BASELINE	PROVENANT	CHANGE
Tool-result tokens	52,432	14,541	−72%
Cumulative input tokens	420,465	184,888	−56%
Task completed	yes	yes	

One scripted workload; tokens estimated as characters ÷ 4.

RED TEAM
Round 2

41 gaps found by round 2: 100 new practical attacks covering cloud and database destruction, credential theft, regulated IDs and non-HTTP exfiltration. **All 41 are fixed** and kept as permanent regression cases.

LIVE CHECKS
Real processes

- pass Allowed command really runs
- pass rm -rf on the workspace never reaches the server
- pass Secret file read is denied, not leaked
- pass Write outside the workspace never lands
- pass Notification-shaped tools/call is dropped
- pass Unknown JSON-RPC method refused
- pass Malformed JSON gets a parse error
- pass Risky call is held; nothing runs
- pass Audit chain verifies after the run

LIMITS
Not proven yet

Paraphrase. Provenance is value-based, so a reworded address can slip that one rule. The other rules still apply.

Shell. Analysis is pattern-based; a script written then run is judged by the run command.

Remote MCP. Only local stdio servers are proxied today.

Field tests. Phone approval of held calls and the Windows one-liner have not run on real infrastructure yet.

Passing 225 cases is evidence for those cases, not proof that no bypass exists.